

Buổi 11: Mảng và lặp qua mảng

Mảng trong JS: khai báo, thêm/xoá phần tử
Các hàm xử lý mảng: for, forEach, map, filter, ...

1. Mảng trong JS: khai báo, thêm/xoá phần tử

1. Giới thiệu về Mảng

✔ Mảng là gì?

Mảng là kiểu dữ liệu đặc biệt dùng để lưu trữ danh sách các giá trị (có thể là số, chuỗi, object, hoặc thậm chí là mảng khác).

Các phần tử trong mảng được đánh chỉ số (index) từ 0.

🔍 Ví dụ:

```
let fruits = ['Táo', 'Chuối', 'Cam'];
console.log(fruits[0]); // Táo
```

✂ 2. Khai báo mảng

✔ Cách khai báo:

```
let arr1 = []; // Mảng rỗng
let arr2 = [1, 2, 3]; // Mảng số
let arr3 = ['a', 'b', 'c']; // Mảng chuỗi
let arr4 = [1, 'abc', true, null]; // Mảng hỗn hợp
```

🔥 Truy cập phần tử:

```
console.log(arr3[1]); // 'b'
```

✚ 3. Thêm phần tử vào mảng

✔ Dùng push() → thêm vào cuối:

```
let numbers = [1, 2];
numbers.push(3);
console.log(numbers); // [1, 2, 3]
```

✔ Dùng unshift() → thêm vào đầu:

```
numbers.unshift(0);
console.log(numbers); // [0, 1, 2, 3]
```

➖ 4. Xoá phần tử khỏi mảng

✔ Dùng pop() → xoá phần tử cuối cùng:

```
numbers.pop();
console.log(numbers); // [0, 1, 2]
```

✔ Dùng shift() → xoá phần tử đầu tiên:

```
numbers.shift();
console.log(numbers); // [1, 2]
```

✔ Dùng splice() → xoá tại vị trí bất kỳ:

```
let colors = ['Đỏ', 'Xanh', 'Vàng'];
colors.splice(1, 1); // xoá 1 phần tử ở vị trí index 1
console.log(colors); // ['Đỏ', 'Vàng']
```

✔ Xoá và thêm cùng lúc:

```
colors.splice(1, 1, 'Tím'); // Xoá 'Vàng', thêm 'Tím'
console.log(colors); // ['Đỏ', 'Tím']
```

🔍 5. Duyệt mảng (giới thiệu thêm)

✔ Dùng for:

```
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}
```

✔ Dùng for...of:

```
for (let color of colors) {
  console.log(color);
}
```

🖌 6. Bài tập

👉 Bài tập 1:

```
// Cho mảng sau:
let nums = [5, 3, 9, 1];
// Thêm số 7 vào đầu mảng
// Xoá số cuối cùng
// Xoá số ở vị trí index = 1
// In mảng sau khi thay đổi
```

📌 Tóm tắt

Hành động	Phương thức
Thêm cuối	<code>.push()</code>
Thêm đầu	<code>.unshift()</code>
Xoá cuối	<code>.pop()</code>
Xoá đầu	<code>.shift()</code>
Xoá tại vị trí	<code>.splice(i, 1)</code>
Thêm/xoá bất kỳ	<code>.splice()</code>

2. Các hàm xử lý mảng: for, forEach, map, filter, ...

👉 1. Giới thiệu

Trong JavaScript, ngoài vòng lặp for, mảng còn có nhiều phương thức (method) giúp xử lý dữ liệu dễ dàng và rõ ràng hơn như:

forEach, map, filter, find, some, every, reduce, v.v.

🔥 2. for (vòng lặp cơ bản)

✔ Cú pháp:

```
for (let i = 0; i < array.length; i++) {
  // xử lý array[i]
}
```

🔍 3. forEach() – Duyệt qua từng phần tử

✔ Cú pháp:

```
array.forEach(function(element, index, array) {
  // xử lý element
});
```

👉 Ví dụ:

```
const fruits = ['Táo', 'Chuối', 'Cam'];
fruits.forEach(function(fruit, index) {
  console.log(index + ': ' + fruit);
});
```

🧠 Ghi nhớ:

- forEach không trả về mảng mới.
- Không dùng được với break hoặc return để thoát vòng lặp.

🔍 4. map() – Tạo mảng mới từ mảng cũ

✔ Cú pháp:

```
const newArray = array.map(function(element, index, array) {
  return newValue;
});
```



👉 Ví dụ:

```
const numbers = [1, 2, 3];
const doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6]
```

🧠 Ghi nhớ:

- map luôn trả về mảng mới.
- Không làm thay đổi mảng gốc.

👉 5. filter() – Lọc ra phần tử thỏa điều kiện

✔ Cú pháp:

```
const result = array.filter(function(element, index, array) {
  return điều_kiện;
});
```

👉 Ví dụ:

```
const numbers = [1, 2, 3, 4, 5];
const even = numbers.filter(n => n % 2 === 0);
console.log(even); // [2, 4]
```

🔍 6. find() – Tìm phần tử đầu tiên thỏa điều kiện

✔ Cú pháp:

```
const result = array.find(function(element) {
  return điều_kiện;
});
```

👉 Ví dụ:

```
const users = [
  { name: 'An', age: 22 },
  { name: 'Binh', age: 30 },
];
const result = users.find(user => user.age > 25);
console.log(result); // { name: 'Binh', age: 30 }
```

✔ 7. some() – Có ít nhất 1 phần tử thỏa điều kiện?

```
const nums = [1, 3, 5];
console.log(nums.some(n => n % 2 === 0)); // false
```

✔ 8. every() – Tất cả phần tử thỏa điều kiện?

```
console.log(nums.every(n => n > 0)); // true
```

📊 9. reduce() – Tính tổng, gộp dữ liệu

✔ Cú pháp:

```
array.reduce(function(accumulator, currentValue) {
  return giá_trị_mới;
}, giá_trị_khởi_đầu);
```

👉 Ví dụ:

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum); // 10
```

📋 10. Tóm tắt bảng so sánh

Hàm	Tác dụng	Trả về gì?
<code>for</code>	Lặp truyền thống	không
<code>forEach</code>	Duyệt mảng	không
<code>map</code>	Tạo mảng mới từ mảng cũ	✔ mảng mới
<code>filter</code>	Lọc ra phần tử thỏa điều kiện	✔ mảng mới
<code>find</code>	Tìm phần tử đầu tiên phù hợp	✔ phần tử
<code>some</code>	Có phần tử nào thỏa?	✔ true/false
<code>every</code>	Tất cả phần tử thỏa?	✔ true/false
<code>reduce</code>	Tính toán, gộp dữ liệu	✔ 1 giá trị

🖌 11. Bài tập

Bài tập 1:

```
// Cho mảng số: [1, 2, 3, 4, 5]
// Yêu cầu:
// - In ra các số chẵn
// - Tạo mảng mới gấp đôi các số
// - Tính tổng của tất cả các số
```

Bài tập 2:

Cho mảng số [1, 2, 3, 4, 5]:

- Dùng for để tính tổng các phần tử.
- Dùng forEach để in ra từng phần tử kèm chỉ số của nó.

Bài tập 3:

- Cho mảng tên ["An", "Binh", "Cường"]:
- Dùng forEach để in ra câu: "Xin chào, <tên>".

Bài tập 4:

Cho mảng số [10, 15, 20, 25, 30]:

- Lọc ra các số chia hết cho 5.
- Lọc ra các số lớn hơn 20.

Bài tập 5:

Cho mảng số [1, 2, 3, 4, 5, 6]:

- Lọc ra các số chẵn, sau đó nhân đôi chúng.
- Tính tổng của kết quả.